C Programming – Assignment 2 (CO2) Report

Name: <YOUR NAME>
Register No.: <YOUR REGISTER NUMBER>
GitHub repository: <https://github.com/<user>/CSA02-<RegisterNumber>-Assignment2>
Compiler: gcc 13.3.0, flags `-Wall -Wextra`. All counts below are real measurements copied from the `output*.txt` files in each folder.

Q1 – Precedence, Associativity and the Discount Bug

Code under test

```
        total = price * qty - price * qty * 15 / 100 * senior + price * qty * 5 / 100;
        net   = total > 1000 ? total - 50 : total;
        count = count++ + 1;
```

Operator order (precedence and associativity)

`*` and `/` have higher precedence than `+` and `-`, and associate left to right. Order for the total expression:

1. `price*qty` (term 1)
2. `price*qty` → `*15` → `/100` → `*senior` (term 2; strictly left to right, so division happens BEFORE multiplying by `senior`)
3. `price*qty` → `*5` → `/100` (term 3)
4. term1 − term2, then + term3 (left to right)
5. `=` last (right-to-left, lowest precedence).

For `net`: `>` binds tighter than `?:`, `?:` binds tighter than `=`; `?:` is right-associative.

Predictions (made before running) and observations

Bill	price	qty	senior	Predicted total	Observed total	Predicted net	Observed net	Intended (correct)
1	250	4	1	1000 − (15000/100=150) + (5000/100=50) = **900**	900	900	900	892.50
2	99	3	1	297 − (4455/100=44) + (1485/100=14) = **267**	267	267	267	265.0725
3	1200	1	0	1200 − 0 + 60 = **1260**	1260	1260 − 50 = **1210**	1210	1210

Which expressions differ from the pharmacist's intent

1. **Logic error (not precedence):** GST (5%) is computed on the *gross* amount, not on the discounted amount. Bill 1: 900 instead of 892.50.
2. **Integer division:** `297*15/100` = 44 (true 44.55) and `297*5/100` = 14 (true 14.85); paise are lost. Bill 2 is 267 instead of 265.0725. Also `15/100*297` evaluates to 0, which shows how the order of operands matters in integer arithmetic.
3. Bill 3 is correct only by coincidence (no discount, and 5% of 1200 is exact).
4. `net`: 50 off is applied on the post-GST total; confirm this with the pharmacist. The ternary itself is correct.

Inquiry – why `count = count++ + 1` is undefined

`count` is modified twice (by `++` and by `=`) between two sequence points, with no ordering defined by the language. The compiler may store the incremented value, the `+1` value, or anything else. gcc -Wall reports: `warning: operation on 'count' may be undefined [-Wsequence-point]` (`Q1/compile_warnings.txt`). With count=5 gcc gave 6 here, but this is not guaranteed on other compilers or optimisation levels.

Design decision – corrected calculation

```
        gross    = price * qty;
        discount = senior ? gross * 0.15 : 0.0;
        taxable  = gross - discount;
        gst      = taxable * 0.05;
        total    = taxable + gst;
        net      = total > 1000.0 ? total - 50.0 : total;
```

Intermediate variables were chosen (brackets alone would still leave one unreadable line and still repeat `price*qty`). `double` removes truncation; in a real billing system integer paise would be preferred to avoid binary floating-point error.

Alternative concept – intermediate variables

	Single expression	Intermediate variables
Arithmetic operations	8 mul/div + 2 add/sub = 10	5 (price*qty computed once)
Extra space	0	5 doubles (40 bytes), O(1)
Time complexity	O(1)	O(1)
Correctness / readability	easy to get wrong; wrong tax base	each step named and testable

An optimising compiler may remove the repeated `price*qty` anyway, so the real benefit is correctness and readability, not speed.

Q2 – Short-Circuit Evaluation

Experiments (observed, `Q2/output.txt`)

Expression	Second operand evaluated?
`0 && f()`	NO
`1 && f()`	YES
`1 || f()`	NO
`0 || f()`	YES
`0 & f()`	YES (& always evaluates both)
`1 | f()`	YES

`(0 && x++)` leaves x = 0; `(0 & x++)` makes x = 1. Also `2 && 1` = 1 but `2 & 1` = 0, so `&` is not a drop-in replacement for `&&` on non-0/1 values.

Call counts: 1000 passengers (305 seniors, 416 eligible class, 102 disability card; fixed LCG seed)

Version	Slow-function calls (no audit)	Audit entries (audit on)
A: `(S && E) || slow()`	**875**	875
B: `slow() || (S && E)`	1000	1000
C: `(S & E) | slow()`	1000	1000
D: nested if	**875**	875

875 = 1000 − 125, where 125 passengers satisfy S && E, so the OR short-circuits before the slow call.

Order of conditions

- For `A || B`, put the condition that is **cheap and most likely TRUE** first; for `A && B`, put the cheap condition that is **most likely FALSE** first. Here, putting the cheap `(S&&E)` first saved 12.5% of the slow calls. Putting `slow()` first would save only when the card is held (10.2%), but you pay for the call every time.
- Order is irrelevant when both operands are equally cheap, have no side effects, and probabilities are similar, or when the result of the first never decides the outcome.

Changed condition – audit log

If every passenger must be logged, short-circuiting is now a **bug**: version A/D log only 875 of 1000 passengers. The verification (with its logging) must be executed unconditionally, e.g. `int card = slow_verify(...); if ((S && E) || card)`, giving 1000 calls and 1000 log entries in any order. Better still: separate logging from verification so the audit does not depend on evaluation order. The recommended order from the unaudited case no longer applies.

Alternative concept – nested if, or `&`/`|`

- Nested `if`: behaves exactly like `&&`/`||`; slow calls = n(1 − p) where p = P(S∧E) (875 measured).
- `&`/`|`: slow function is called for all n passengers: exactly n calls (1000 measured).
- Time: both O(n) worst case; short-circuit reduces only the constant (measured 875 vs 1000). Space: O(1) for all.

Q3 – Device-Status Register with Bitwise Operators

Complete the code (TODO 1–4, `Q3/q3_complete.c`)

```
        switch_on : return (unsigned char)(reg |  (1u << dev));
        switch_off: return (unsigned char)(reg & ~(1u << dev));
        toggle    : return (unsigned char)(reg ^  (1u << dev));
        is_on     : return (reg >> dev) & 1u;
```

Logic: `1u << dev` builds a mask with only bit `dev` set. OR sets it, AND with the complement clears it, XOR flips it, and shifting right then masking with 1 reads it.

Register after each statement of the given `main()` (predicted, then verified):

Statement	Register
`reg = 0`	00000000
`switch_on(LIGHT)`	00000001
`switch_on(AC)`	00000101 ← printed
`toggle(FAN)`	00000111
`switch_off(LIGHT)`	00000110 ← printed

Output: `00000101`, `00000110`, `AC is ON, PUMP is OFF`, which matches the expected output.

Own designed sequence (prediction vs observation, `Q3/output_extra.txt`)

on LIGHT 00000001 → on FAN 00000011 → on PUMP 00010011 → toggle AC 00010111 → off FAN 00010101 → toggle LIGHT 00010100 → toggle HEATER 00011100. All 7 predictions matched. Devices on = 3 (loop count and Kernighan's `r &= r-1`, 3 iterations).

Inquiry

- `reg & 1 << dev` parses as `reg & (1 << dev)` because `<<` has higher precedence than `&`, so it tests bit `dev`. `(reg & 1) << dev` tests only bit 0 and then shifts the answer. With reg = 0x14, dev = 2: first gives 4 (correct), second gives 0. With reg = 0x15 (bit 0 set, bit 2 clear): second gives 4 even though bit 2 is not tested.
- With a **signed char** and bit 7 set (0x80) the value is −128. `>>` on a negative value is implementation-defined (gcc: arithmetic shift, sign extension): `sc >> 7` = −1, `sc >> 4` = −8 (0xFFFFFFF8 as int), whereas for unsigned char `>> 7` = 1. Also `sc == 0x80` is false (−128 vs 128). `~` promotes to int, so results need masking with `0xFF`. Hence `unsigned char` is correct for a bit register.

Alternative – `char state[8]`

	Bit register	Array
set / clear / toggle / test	O(1)	O(1)
Count ON devices	O(k) with Kernighan (k = devices ON) or O(8)	O(8)
Space per home	1 byte	8 bytes
Space, 1,000,000 homes	**1,000,000 bytes**	**8,000,000 bytes**

Measured count over 1e6 homes: bit/Kernighan 3,999,996 iterations, 8.6 ms; array 8,000,000 iterations, 19.9 ms (timings vary by machine). Trade-off: array is simpler to read and index; bits are 8× smaller and can be copied/compared/saved as one byte but need care with precedence and signedness.

Q4 – else-if Ladder or switch

Complete the code (TODO 1)

```
        switch (marks / 10) {
        case 10:
        case 9:  return 'S';
        case 8:  return 'A';
        case 7:  return 'B';
        case 6:  return 'C';
        case 5:  return 'D';
        default: return 'F';
        }
```

Cases 10 and 9 are combined (100 and 90–99 are both S); `default` covers 0–49. Output: `100:S 95:S 83:A 50:D 49:F -5:?` (matches expected).

Test results (`Q4/output_compare.txt`)

marks	ladder	switch	table	switch without break
100	S	S	S	S
90	S	S	S	S
89	A	A	A	**B**
80	A	A	A	**B**
79 / 70	B	B	B	B
69 / 60	C	C	C	C
59 / 50	D	D	D	D
49 / 0	F	F	F	F
−1 / 101	?	?	?	?

Exhaustive check −5..105 (111 values): ladder, switch and table disagree **0** times.

Missing break

Removing the `return`/`break` after case 8 lets execution fall into case 7: all marks 80–89 become **B** (10 wrong values out of 111).

Questions

- **Why can switch not test 75–84?** Each `case` label must be a single integer constant; a range is not allowed in standard C (the `case 75 ... 84` form is a non-portable gcc extension). Also `marks/10` merges 70–79 into one case, so 75–84 straddles two cases.
- **Why the range check comes first:** in C, `-5 / 10` is 0 (truncation toward zero), so without the check −9..−1 would be graded F.
- **Easiest to change when boundaries change:** the lookup table or the ladder (one number to edit). The `marks/10` switch silently assumes boundaries are multiples of 10, so a boundary such as 75 forces a redesign.

Boundary-exposing test set

For every boundary b ∈ {0, 50, 60, 70, 80, 90, 100}: test b−1, b, b+1; plus −1, 101, and extremes (INT_MIN, INT_MAX) for the range guard.

Design decision – scholarship (grade × income)

A **2-D lookup table** `scholarship[grade][income band]` is the best structure: the rule is data, not logic, so changing amounts needs no code change, every cell is visible and testable, and lookup is O(1). Nested if grows quickly (5 × 4 branches), and switch-inside-if still needs a second decision per grade. Implemented in `q4_compare.c` and printed as a table.

Alternative – lookup table

Table `char grades[11]` indexed by `marks/10`: O(1) time, 11 bytes of space. The ladder took 502 comparisons for marks 0..100 (4.97 on average, maximum 6); switch uses one jump-table index (O(1) when the compiler generates a jump table) and table lookup is a single index. Table is the shortest and easiest to modify, but it has the same multiple-of-10 limitation as the switch.

Q5 – Counting the Work Done by a Loop

Deriving the loop condition

If n = a·b with a ≤ b, then a² ≤ a·b = n, so the smaller divisor satisfies a ≤ √n. Hence only divisors i with **`i * i <= n`** need testing (no `sqrt()` call). Stop with `return 0` at the first divisor; counting `checks++` before each `%`.

Complete the code (`Q5/q5_complete.c`)

```
        for (unsigned long i = 2; i * i <= n; i++) {
            checks++;
            if (n % i == 0) return 0;
        }
        return 1;
```

Output matches expected exactly: 97 → 8, 100 → 1, 7919 → 87, 1000003 → 999.

Division counts (all versions, `Q5/output_versions.txt`)

n	result	`i < n` (for)	`i*i <= n` for	while	do-while
2	prime	0	0	0	0
3	prime	1	0	0	0
4	not prime	1	1	1	1
97	prime	95	8	8	8
100	not prime	1	1	1	1
7919	prime	7917	87	87	87
1000003	prime	1000001	999	999	999
999983	prime	999981	998	998	998
1000001 (= 101 × 9901)	not prime	100	100	100	100

Inquiry

- **do-while for n = 2 and 3:** a do-while executes the body once before checking the condition. For n = 2, the first pass tests `2 % 2 == 0` and wrongly reports "not prime" (observed). For n = 3 the result is correct but one unnecessary division is done (3 % 2). So the do-while version needs a guard (`if (n < 4) return 1;`) *before* the loop.
- **Overflow of `i * i <= n`:** with 32-bit unsigned, n = 4294967291 (prime), `i*i` wraps to 0 when i = 65536, so the condition stays true and the loop never ends correctly (stopped by the safety limit in the experiment). Fix: write `i <= n / i`, which does not multiply (verified: reports prime correctly). With 64-bit `unsigned long` the same problem arises for n near 2^64.

Alternative – Sieve of Eratosthenes (N = 1,000,003)

	Trial division (√n)	Sieve
Time	O(√n) per query	O(N log log N) once (2,122,051 marking operations measured, 4.9 ms), then O(1) per query
Space	O(1)	O(N): 1,000,004 bytes
Divisions	999 for the large prime	0

Trial division of every n from 2 to 1,000,003: **67,741,504 divisions** in total. Sieve found 78,499 primes. Sieve is better for many queries over a known range; trial division is better for one-off or very large n.

Q6 – Loan Repayment Schedule

Design

- **Loop choice: `while`.** The number of months is unknown in advance, and if the principal is already 0 the body must run zero times, which `do-while` cannot do. `for` fits only if the count is known.
- **Termination:** `while (balance > 0.005)`. `balance != 0` is dangerous because floating-point values rarely land exactly on 0: with repayments subtracted but no final-month adjustment, the balance went to −1354.62 and never equalled 0; repeated addition of 0.1 never hits 1.0 exactly (`b = 2.1000000000000005` after 20 steps).
- **Last month:** `payment = (balance < pay) ? balance : pay`; so the last instalment pays only what remains.
- **Repayment ≤ interest:** the balance never falls. The program rejects this before the loop (`pay <= principal * rate`).

Hand-verified schedule: Rs 10,000, 2% per month, Rs 2,000 repayment (program output equals hand calculation)

Month	Opening	Interest (2%)	Payment	Closing
1	10000.00	200.00	2000.00	8200.00
2	8200.00	164.00	2000.00	6364.00
3	6364.00	127.28	2000.00	4491.28
4	4491.28	89.83	2000.00	2581.11
5	2581.11	51.62	2000.00	632.73
6	632.73	12.65	645.38	0.00

6 months, total interest Rs 645.38, total paid Rs 10,645.38.

Growth investigation (P = 10,000; monthly interest = 200)

Repayment	Months (simulated)	Closed form n	Total interest
5000	3	2.06	310.08
3000	4	3.48	459.50
2000	6	5.32	645.38
1000	12	11.27	1270.33
500	26	25.80	2898.73
300	56	55.48	6644.17
250	82	81.27	10318.98
220	122	121.09	16639.92
210	154	153.74	22286.50
205	188	187.53	28444.00
201	268	267.81	43829.75
200.01	> 360 (limit hit)	500.11	–
200 / 150	rejected	–	–

Prediction: months grow slowly while the repayment is well above the interest, then explode as repayment → 200, because n = −ln(1 − rP/E)/ln(1+r) → ∞ as E → rP. The measurements confirm this (201 → 268 months, 200.01 → over 360).

Deployment decision

Safety limit = 360 months (30 years) via `break`/return; the message when reached: "*Loan not repayable within 360 months at this instalment; increase repayment.*" A hard failure message is better than printing a schedule that goes on forever, and the up-front check `repayment <= interest` gives a more precise message in the impossible case.

Alternative – closed formula

n = −ln(1 − rP/E) / ln(1 + r) → 5.32, so ceil = **6** (matches the simulation). | | Simulation loop | Formula | |---|---|---| | Time | O(n) iterations (6 here, up to 360) | O(1) | | Space | O(1) (O(n) if schedule stored) | O(1) | | Gives monthly schedule? | yes | no, only the count (not the last payment unless recomputed) | | Precision | small rounding accumulation each month | argument 1 − rP/E → 0 near E = rP (loss of precision); NaN/invalid if E ≤ rP so it must be tested first |

Use the formula for a quick feasibility estimate, and the loop for the actual schedule.

Q7 – Day-4 Design Challenge: Redesign of Q5 (prime verification)

A. Diagnose the bottleneck (evidence from Q5)

- Operations are all `%` divisions in the loop; the count grows as √n: 999 for 1000003, 87 for 7919, 8 for 97.
- Verifying every ticket 2..1,000,003 costs **67,741,504 divisions**.
- Work is wasted in two ways: (1) after testing 2, every other even divisor is useless, and after 3 so are multiples of 3, so only 1/3 of the tested divisors can ever succeed; (2) divisibility information is recomputed for each ticket, although the tickets share the same small primes.

B. Objective

(1) Cut divisions for large prime tickets by at least 60% without extra memory (target ≤ 400 for 1,000,003); (2) for the batch workload, remove per-ticket divisions altogether.

C. Redesign

Redesign 1 – 6k ± 1 wheel: every prime above 3 has the form 6k ± 1, so test 2, 3, and then 5, 7, 11, 13, 17, 19, … (step 6, two tests per step). No extra memory.

```
function is_prime(n):
    if n < 2: return 0
    if n < 4: return 1
    if n mod 2 == 0 or n mod 3 == 0: return 0
    i = 5
    while i*i <= n:
        if n mod i == 0 or n mod (i+2) == 0: return 0
        i = i + 6
    return 1
```

```
 [start] -> n<2? --yes--> 0
            n<4? --yes--> 1
            n%2==0 or n%3==0? --yes--> 0
            i=5 -> i*i<=n? --no--> 1
                    n%i==0 or n%(i+2)==0? --yes--> 0
                    i+=6 -> (back to i*i<=n)
```

Redesign 2 – Sieve of Eratosthenes (batch): mark composites once, then answer each ticket by array lookup. Costs N+1 bytes and a one-time build.

D. Prediction (made before implementation)

Redesign 1: only about 1/3 of the original divisor candidates remain, so ≈ √n/3 ≈ 333 + 2 divisions for 1,000,003 (a ~66% reduction) and about 65% fewer divisions over all tickets. Redesign 2: ≈ N·ln ln N ≈ 2–3 million marking operations in total, about 30× fewer than 67.7 million divisions, with 1 MB of memory.

E. Implementation and measurements (same inputs, `Q7/output.txt`)

n	original	6k±1	saved	sieve
97	8	4	50.0%	0
100	1	1	0%	0
7919	87	30	65.5%	0
1000003	999	334	66.6%	0
999983	998	334	66.5%	0
1000001	100	35	65.0%	0

Batch of every ticket 1..1,000,003: | Method | Divisions | Time | Extra memory | |---|---|---|---| | Original | 67,741,504 | 180.1 ms | 0 | | 6k±1 | 23,739,568 (−65.0%) | 58.6 ms | 0 | | Sieve | 0 (2,122,051 marks) | 5.0 ms build + 1.7 ms lookups | 1,000,004 bytes |

The wheel and sieve agree on all 78,499 primes (0 disagreements).

Few-query break-even (large n near 1,000,003): 1 query: original 999, wheel 334 vs sieve 2,122,051 marks; 10,000 queries: original 1,872,978, wheel 640,850, sieve 2,122,051 marks.

F. Defence of the final design

- **Objective met:** 66.6% fewer divisions for the large prime (334 ≤ 400) and 65.0% fewer over the batch, matching the 66%/65% prediction. For batch verification the sieve removes the divisions entirely (about 32× fewer basic operations, about 27× faster in the measured run).
- **Costs:** the wheel adds slightly more complex code and the special-case guards for 2 and 3; the sieve needs O(N) memory (1 MB here) and a known upper bound.

- **When the original is still preferable:** a single or very few queries (the sieve's 2.1 million marks versus 334 divisions per wheel query), very large n beyond the memory limit, or when simple code matters more than speed. The 6k±1 wheel does not improve the O(√n) complexity, only the constant factor.
- **Final recommendation:** use the sieve for verifying all tickets up to 1,000,003 (organisers' use case), and the 6k±1 test for isolated checks. Timings are single-run measurements and vary by machine; operation counts are the primary evidence.

Appendix – How to reproduce

See `README.md` for compile/run commands; all raw outputs are in each `Qn/output*.txt`.

Appendix A – C Source Code and Raw Program Output

Q1

Q1/q1_billing.c

```c
/* Q1 - Precedence, associativity and the discount bug (Jan Aushadhi billing) */
#include <stdio.h>

/* Developer's original single expression (all int arithmetic). */
static void buggy(int price, int qty, int senior, int *total, int *net)
{
    *total = price * qty - price * qty * 15 / 100 * senior + price * qty * 5 / 100;
    *net = *total > 1000 ? *total - 50 : *total; /* Rs 50 off above Rs 1000 */
}

/* Corrected: intermediate variables, double arithmetic, GST on discounted amount. */
static void correct(double price, int qty, int senior, double *gross, double *discount,
                    double *taxable, double *gst, double *total, double *net)
{
    *gross    = price * qty;
    *discount = senior ? *gross * 0.15 : 0.0;
    *taxable  = *gross - *discount;
    *gst      = *taxable * 0.05;
    *total    = *taxable + *gst;
    *net      = (*total > 1000.0) ? *total - 50.0 : *total;
}

int main(void)
{
    struct { int price, qty, senior; } bills[] = {
        {250, 4, 1},   /* senior, gross 1000          */
        {99, 3, 1},    /* senior, odd amount (truncation) */
        {1200, 1, 0}   /* non-senior, above Rs 1000   */
    };

    printf("%-5s %-6s %-4s %-7s | %-10s %-8s | %-8s %-8s %-8s %-9s %-9s\n",
           "price", "qty", "sen", "", "buggy_tot", "buggy_net",
           "gross", "disc", "taxable", "total", "net");
    for (int i = 0; i < 3; i++) {
        int bt, bn;
        double g, d, t, gs, tot, n;
        buggy(bills[i].price, bills[i].qty, bills[i].senior, &bt, &bn);
        correct(bills[i].price, bills[i].qty, bills[i].senior, &g, &d, &t, &gs, &tot, &n);
        printf("%-5d %-6d %-4d %-7s | %-10d %-9d | %-8.2f %-8.2f %-8.2f %-9.4f %-9.4f\n",
               bills[i].price, bills[i].qty, bills[i].senior, "", bt, bn, g, d, t, tot, n);
    }

    /* Integer-division demonstration */
    printf("\nInteger division: 297*15/100 = %d (true 44.55), 297*5/100 = %d (true 14.85)\n",
           297 * 15 / 100, 297 * 5 / 100);
    printf("Order matters: 15/100*297 = %d (zero!), 297*15/100 = %d\n", 15 / 100 * 297, 297 * 15 / 100);

    /* Third statement: undefined behaviour (count modified twice, no sequence point) */
    int count = 5;
    count = count++ + 1;
    printf("\ncount = count++ + 1 with count=5 gave %d (undefined behaviour; compiler dependent)\n", count);
    return 0;
}
```

Output: **Q1/compile_warnings.txt**

```
Q1/q1_billing.c: In function 'main':
Q1/q1_billing.c:50:11: warning: operation on 'count' may be undefined [-Wsequence-point]
   50 |     count = count++ + 1;
      |     ~~~~~~^~~~~~~~~~~~
```

Output: **Q1/output.txt**

```
price qty    sen          | buggy_tot  buggy_net | gross    disc     taxable  total     net
250   4      1            | 900        900       | 1000.00  150.00   850.00   892.5000  892.5000
99    3      1            | 267        267       | 297.00   44.55    252.45   265.0725  265.0725
1200  1      0            | 1260       1210      | 1200.00  0.00     1200.00  1260.0000 1210.0000

Integer division: 297*15/100 = 44 (true 44.55), 297*5/100 = 14 (true 14.85)
Order matters: 15/100*297 = 0 (zero!), 297*15/100 = 44

count = count++ + 1 with count=5 gave 6 (undefined behaviour; compiler dependent)
```

Q2

Q2/q2_shortcircuit.c

```c
/* Q2 - Short-circuit evaluation: railway concession */
#include <stdio.h>

#define N 1000

static unsigned long slow_calls = 0;     /* calls to slow verification function */
static unsigned long audit_log_len = 0;  /* entries in audit log (changed condition) */
static int audit_enabled = 0;

static int slow_verify(int disability_card)
{
    slow_calls++;
    if (audit_enabled) audit_log_len++;
    return disability_card;
}

static int cheap_calls = 0;
static int cheap(int v) { cheap_calls++; return v; }

static unsigned int seed = 12345;
static int rnd(void) { seed = seed * 1103515245u + 12345u; return (int)((seed >> 16) & 0x7fff); }

static int senior[N], eligible[N], disab[N];

typedef int (*rule_fn)(int i);

/* A: cheap condition first, slow last */
static int rule_A(int i) { return (senior[i] && eligible[i]) || slow_verify(disab[i]); }
/* B: slow first */
static int rule_B(int i) { return slow_verify(disab[i]) || (senior[i] && eligible[i]); }
/* C: non-short-circuit operators: everything always evaluated */
static int rule_C(int i) { return (senior[i] & eligible[i]) | slow_verify(disab[i]); }
/* D: nested ifs (same behaviour as A) */
static int rule_D(int i)
{
    if (senior[i]) { if (eligible[i]) return 1; }
    if (slow_verify(disab[i])) return 1;
    return 0;
}

static void run(const char *name, rule_fn f)
{
    unsigned long granted = 0;
    slow_calls = 0; audit_log_len = 0;
    for (int i = 0; i < N; i++) granted += (unsigned long)f(i);
    printf("  %-28s slow calls = %4lu   audit entries = %4lu   concessions = %lu\n",
           name, slow_calls, audit_log_len, granted);
}

int main(void)
{
    /* ---- Part 1: when is the second operand not evaluated? ---- */
    printf("Part 1: operand evaluation\n");
    int r;
    cheap_calls = 0; r = cheap(0) && cheap(1);
    printf("  0 && f(): result=%d, f called %s (calls=%d)\n", r, cheap_calls == 2 ? "YES" : "NO", cheap_calls);
    cheap_calls = 0; r = cheap(1) && cheap(1);
    printf("  1 && f(): result=%d, f called %s (calls=%d)\n", r, cheap_calls == 2 ? "YES" : "NO", cheap_calls);
    cheap_calls = 0; r = cheap(1) || cheap(0);
    printf("  1 || f(): result=%d, f called %s (calls=%d)\n", r, cheap_calls == 2 ? "YES" : "NO", cheap_calls);
    cheap_calls = 0; r = cheap(0) || cheap(0);
    printf("  0 || f(): result=%d, f called %s (calls=%d)\n", r, cheap_calls == 2 ? "YES" : "NO", cheap_calls);
    cheap_calls = 0; r = cheap(0) & cheap(1);
    printf("  0 &  f(): result=%d, f called %s (calls=%d)  <- & always evaluates both\n", r, cheap_calls == 2 ? "YES" : "
cheap_calls);
    cheap_calls = 0; r = cheap(1) | cheap(0);
    printf("  1 |  f(): result=%d, f called %s (calls=%d)  <- | always evaluates both\n", r, cheap_calls == 2 ? "YES" : "
cheap_calls);

    int x = 0; r = (0 && x++); printf("  (0 && x++): x stays %d ; ", x);
    x = 0; r = (0 & x++); printf("(0 & x++): x becomes %d\n", x);
    (void)r;

    /* & versus && on non-boolean values */
    printf("  2 && 1 = %d, but 2 & 1 = %d (bitwise on non-0/1 values differs)\n", 2 && 1, 2 & 1);

    /* ---- Part 2: 1000 passengers ---- */
    int ns = 0, ne = 0, nd = 0;
    for (int i = 0; i < N; i++) {
        senior[i]   = rnd() % 100 < 30;   /* 30% seniors            */
        eligible[i] = rnd() % 100 < 40;   /* 40% eligible class     */
```

```
            disab[i]    = rnd() % 100 < 10;   /* 10% hold disability card */
            ns += senior[i]; ne += eligible[i]; nd += disab[i];
        }
        printf("\nPart 2: %d passengers (seniors=%d, eligible class=%d, disability=%d)\n", N, ns, ne, nd);
        audit_enabled = 0;
        printf("Without audit log:\n");
        run("A: (S&&E) || slow", rule_A);
        run("B: slow || (S&&E)", rule_B);
        run("C: (S&E) | slow", rule_C);
        run("D: nested if", rule_D);

        audit_enabled = 1;
        printf("With audit log (every passenger MUST be logged):\n");
        run("A: (S&&E) || slow", rule_A);
        run("B: slow || (S&&E)", rule_B);
        run("C: (S&E) | slow", rule_C);
        run("D: nested if", rule_D);
        return 0;
    }
```

Output: `Q2/output.txt`

```
Part 1: operand evaluation
  0 && f(): result=0, f called NO (calls=1)
  1 && f(): result=1, f called YES (calls=2)
  1 || f(): result=1, f called NO (calls=1)
  0 || f(): result=0, f called YES (calls=2)
  0 &  f(): result=0, f called YES (calls=2)  <- & always evaluates both
  1 |  f(): result=1, f called YES (calls=2)  <- | always evaluates both
  (0 && x++): x stays 0 ; (0 & x++): x becomes 1
  2 && 1 = 1, but 2 & 1 = 0 (bitwise on non-0/1 values differs)

Part 2: 1000 passengers (seniors=305, eligible class=416, disability=102)
Without audit log:
  A: (S&&E) || slow            slow calls =  875   audit entries =    0   concessions = 217
  B: slow || (S&&E)            slow calls = 1000   audit entries =    0   concessions = 217
  C: (S&E) | slow              slow calls = 1000   audit entries =    0   concessions = 217
  D: nested if                 slow calls =  875   audit entries =    0   concessions = 217
With audit log (every passenger MUST be logged):
  A: (S&&E) || slow            slow calls =  875   audit entries =  875   concessions = 217
  B: slow || (S&&E)            slow calls = 1000   audit entries = 1000   concessions = 217
  C: (S&E) | slow              slow calls = 1000   audit entries = 1000   concessions = 217
  D: nested if                 slow calls =  875   audit entries =  875   concessions = 217
```

Q3

`Q3/q3_complete.c`

```
    #include <stdio.h>
    /* Each bit of one byte stores the ON/OFF state of one device. */
    enum { LIGHT = 0, FAN = 1, AC = 2, HEATER = 3, PUMP = 4 };
    unsigned char switch_on(unsigned char reg, int dev)
    {
        return (unsigned char)(reg | (1u << dev));      /* TODO 1: set bit */
    }
    unsigned char switch_off(unsigned char reg, int dev)
    {
        return (unsigned char)(reg & ~(1u << dev));     /* TODO 2: clear bit */
    }
    unsigned char toggle(unsigned char reg, int dev)
    {
        return (unsigned char)(reg ^ (1u << dev));      /* TODO 3: flip bit */
    }
    int is_on(unsigned char reg, int dev)
    {
        return (reg >> dev) & 1u;                       /* TODO 4: test bit */
    }
    void show(unsigned char reg)
    {
        for (int bit = 7; bit >= 0; bit--) putchar(((reg >> bit) & 1u) ? '1' : '0');
        putchar('\n');
    }
    int main(void)
    {
        unsigned char reg = 0;
        reg = switch_on(reg, LIGHT);
        reg = switch_on(reg, AC);
        show(reg);
        reg = toggle(reg, FAN);
        reg = switch_off(reg, LIGHT);
        show(reg);
        printf("AC is %s, PUMP is %s\n", is_on(reg, AC) ? "ON" : "OFF",
               is_on(reg, PUMP) ? "ON" : "OFF");
```

```
            return 0;
        }
```

Q3/q3_extra.c

```
        /* Q3 extras: register trace, popcount, precedence, signedness, array alternative */
        #include <stdio.h>
        #include <time.h>
        #include <string.h>

        enum { LIGHT = 0, FAN = 1, AC = 2, HEATER = 3, PUMP = 4 };
        static unsigned char switch_on(unsigned char r, int d)  { return (unsigned char)(r | (1u << d)); }
        static unsigned char switch_off(unsigned char r, int d) { return (unsigned char)(r & ~(1u << d)); }
        static unsigned char toggle(unsigned char r, int d)     { return (unsigned char)(r ^ (1u << d)); }
        static int is_on(unsigned char r, int d)                { return (r >> d) & 1u; }
        static void show(unsigned char r) { for (int b = 7; b >= 0; b--) putchar(((r >> b) & 1u) ? '1' : '0'); }

        static int count_on_loop(unsigned char r) { int c = 0; for (int b = 0; b < 8; b++) c += (r >> b) & 1u; return c; }
        static int count_on_kernighan(unsigned char r, int *steps) { int c = 0; *steps = 0; while (r) { r &= (unsigned char)(r -
c++; (*steps)++; } return c; }

        int main(void)
        {
            /* Designed sequence with predictions (written before running):
               start 00000000
               on LIGHT -> 00000001, on FAN -> 00000011, on PUMP -> 00010011,
               toggle AC -> 00010111, off FAN -> 00010101, toggle LIGHT -> 00010100,
               toggle HEATER -> 00011100, on bit7 style value via reg|0x80 -> 10011100   */
            const char *pred[] = {"00000001","00000011","00010011","00010111","00010101","00010100","00011100"};
            unsigned char reg = 0;
            printf("Register trace (predicted vs observed)\n");
            struct { const char *txt; int op; int dev; } seq[] = {
                {"on LIGHT", 0, LIGHT}, {"on FAN", 0, FAN}, {"on PUMP", 0, PUMP}, {"toggle AC", 2, AC},
                {"off FAN", 1, FAN}, {"toggle LIGHT", 2, LIGHT}, {"toggle HEATER", 2, HEATER}};
            for (int i = 0; i < 7; i++) {
                reg = seq[i].op == 0 ? switch_on(reg, seq[i].dev) :
                      seq[i].op == 1 ? switch_off(reg, seq[i].dev) : toggle(reg, seq[i].dev);
                char buf[9]; for (int b = 7; b >= 0; b--) buf[7 - b] = ((reg >> b) & 1u) ? '1' : '0'; buf[8] = 0;
                printf("  %-14s predicted %s observed %s  %s  (0x%02X)\n", seq[i].txt, pred[i], buf,
                       strcmp(pred[i], buf) == 0 ? "OK" : "MISMATCH", reg);
            }
            int steps;
            int k_count = count_on_kernighan(reg, &steps);
            printf("Devices ON (loop)=%d, (Kernighan)=%d in %d iterations\n", count_on_loop(reg), k_count, steps);
            printf("is_on checks: LIGHT=%d FAN=%d AC=%d HEATER=%d PUMP=%d\n",
                   is_on(reg, LIGHT), is_on(reg, FAN), is_on(reg, AC), is_on(reg, HEATER), is_on(reg, PUMP));

            /* Precedence: reg & 1 << dev  ==  reg & (1 << dev)  (shift binds tighter than &)
                           (reg & 1) << dev  tests only bit 0 and then shifts */
            unsigned char r = 0x14; int dev = 2;  /* 00010100: bit 2 set, bit 0 clear */
            printf("\nreg=0x14 dev=2: reg & 1 << dev = %d (tests bit 2: correct, value 4)\n", r & 1 << dev);
            printf("                (reg & 1) << dev = %d (tests bit 0, then shifts: wrong)\n", (r & 1) << dev);
            r = 0x15; /* bit 0 set */
            printf("reg=0x15 dev=2: reg & 1 << dev = %d, (reg & 1) << dev = %d (reports 4 for bit 0, not bit 2)\n", r & 1 << dev,
1) << dev);

            /* Signedness: bit 7 set in signed char */
            signed char sc = (signed char)0x80;  /* implementation-defined conversion, -128 on gcc */
            unsigned char uc = 0x80;
            printf("\nsigned char 0x80 = %d, >>7 gives %d (arithmetic shift, sign extension); unsigned char 0x80 >>7 gives %d\n",
                   sc, sc >> 7, uc >> 7);
            printf("sc >> 7 & 1 = %d (ok after masking), but printing (sc >> 4) = %d, 0x%X (sign-extended)\n", (sc >> 7) & 1, sc
(unsigned)(sc >> 4));
            printf("sc == 0x80 is %s (promotes to -128 vs 128); uc == 0x80 is %s\n", sc == 0x80 ? "true" : "false", uc == 0x80 ?
: "false");
            printf("~sc & 0xFF = 0x%X; ~uc = 0x%X (int promotion: mask result)\n", ~sc & 0xFF, ~uc);

            /* Alternative: char state[8] */
            char st[8] = {0};
            st[LIGHT] = 1; st[AC] = 1; st[FAN] ^= 1; st[LIGHT] = 0;
            printf("\nArray version after same main() steps: ");
            for (int i = 7; i >= 0; i--) putchar('0' + st[i]);
            int cnt = 0, iters = 0; for (int i = 0; i < 8; i++) { cnt += st[i]; iters++; }
            printf("  count ON=%d in %d iterations\n", cnt, iters);

            printf("Space for 1,000,000 homes: bitfield = %zu bytes, array = %zu bytes\n",
                   (size_t)1000000 * sizeof(unsigned char), (size_t)1000000 * sizeof(st));

            /* Timing: count ON devices over 1e6 homes */
            enum { H = 1000000 };
            static unsigned char regs[H]; static char arr[H][8];
            for (int i = 0; i < H; i++) { regs[i] = (unsigned char)((i * 37u) & 0xFF); for (int b = 0; b < 8; b++) arr[i][b] = (r
>> b) & 1; }
            long tot1 = 0, tot2 = 0, it1 = 0, it2 = 0;
            clock_t t0 = clock();
```

```
            for (int i = 0; i < H; i++) { int s; tot1 += count_on_kernighan(regs[i], &s); it1 += s; }
            clock_t t1 = clock();
            for (int i = 0; i < H; i++) for (int b = 0; b < 8; b++) { tot2 += arr[i][b]; it2++; }
            clock_t t2 = clock();
            printf("Counting ON devices for 1e6 homes: bit/Kernighan total=%ld iterations=%ld time=%.1f ms; array total=%ld
iterations=%ld time=%.1f ms\n",
                   tot1, it1, 1000.0 * (t1 - t0) / CLOCKS_PER_SEC, tot2, it2, 1000.0 * (t2 - t1) / CLOCKS_PER_SEC);
            (void)show;
            return 0;
        }
```

Output: `Q3/output_complete.txt`

```
00000101
00000110
AC is ON, PUMP is OFF
```

Output: `Q3/output_extra.txt`

```
Register trace (predicted vs observed)
  on LIGHT       predicted 00000001 observed 00000001  OK  (0x01)
  on FAN         predicted 00000011 observed 00000011  OK  (0x03)
  on PUMP        predicted 00010011 observed 00010011  OK  (0x13)
  toggle AC      predicted 00010111 observed 00010111  OK  (0x17)
  off FAN        predicted 00010101 observed 00010101  OK  (0x15)
  toggle LIGHT   predicted 00010100 observed 00010100  OK  (0x14)
  toggle HEATER  predicted 00011100 observed 00011100  OK  (0x1C)
Devices ON (loop)=3, (Kernighan)=3 in 3 iterations
is_on checks: LIGHT=0 FAN=0 AC=1 HEATER=1 PUMP=1

reg=0x14 dev=2: reg & 1 << dev = 4 (tests bit 2: correct, value 4)
                (reg & 1) << dev = 0 (tests bit 0, then shifts: wrong)
reg=0x15 dev=2: reg & 1 << dev = 4, (reg & 1) << dev = 4 (reports 4 for bit 0, not bit 2)

signed char 0x80 = -128, >>7 gives -1 (arithmetic shift, sign extension); unsigned char 0x80 >>7 gives 1
sc >> 7 & 1 = 1 (ok after masking), but printing (sc >> 4) = -8, 0xFFFFFFF8 (sign-extended)
sc == 0x80 is false (promotes to -128 vs 128); uc == 0x80 is true
~sc & 0xFF = 0x7F; ~uc = 0xFFFFFF7F (int promotion: mask result)

Array version after same main() steps: 00000110  count ON=2 in 8 iterations
Space for 1,000,000 homes: bitfield = 1000000 bytes, array = 8000000 bytes
Counting ON devices for 1e6 homes: bit/Kernighan total=3999996 iterations=3999996 time=8.6 ms; array total=3999996
iterations=8000000 time=19.9 ms
```

Q4

`Q4/q4_compare.c`

```c
        /* Q4: ladder vs switch vs lookup table, boundary tests, missing-break effect, scholarship rule */
        #include <stdio.h>

        static unsigned long cmp_ladder = 0;

        static char grade_ladder(int m)
        {
            cmp_ladder++; if (m < 0 || m > 100) return '?';
            cmp_ladder++; if (m >= 90) return 'S';
            cmp_ladder++; if (m >= 80) return 'A';
            cmp_ladder++; if (m >= 70) return 'B';
            cmp_ladder++; if (m >= 60) return 'C';
            cmp_ladder++; if (m >= 50) return 'D';
            return 'F';
        }

        static char grade_switch(int m)
        {
            if (m < 0 || m > 100) return '?';
            switch (m / 10) {
            case 10: case 9: return 'S';
            case 8: return 'A';
            case 7: return 'B';
            case 6: return 'C';
            case 5: return 'D';
            default: return 'F';
            }
        }

        /* Same switch with the 'return' of case 8 removed (missing break) -> falls through */
        static char grade_switch_nobreak(int m)
        {
            char g = 'F';
            if (m < 0 || m > 100) return '?';
```

```
            switch (m / 10) {
            case 10: case 9: g = 'S'; break;
            case 8: g = 'A';            /* <-- break missing */
            case 7: g = 'B'; break;
            case 6: g = 'C'; break;
            case 5: g = 'D'; break;
            default: g = 'F';
            }
            return g;
        }

        static const char table[11] = {'F','F','F','F','F','D','C','B','A','S','S'};
        static char grade_table(int m)
        {
            if (m < 0 || m > 100) return '?';
            return table[m / 10];
        }

        /* Scholarship: grade x income band -> percentage, via lookup table */
        static int income_band(int income)   /* 0: <2.5L, 1: 2.5-5L, 2: 5-8L, 3: >8L (rupees per year) */
        {
            if (income < 250000) return 0;
            if (income < 500000) return 1;
            if (income < 800000) return 2;
            return 3;
        }
        static int grade_index(char g) { switch (g) { case 'S': return 0; case 'A': return 1; case 'B': return 2; case 'C': retur
case 'D': return 4; default: return -1; } }
        static const int scholarship[5][4] = {
            /* income band: 0    1    2   3 */
            /* S */ {100, 75, 50, 25},
            /* A */ { 75, 50, 25, 0},
            /* B */ { 50, 25, 0, 0},
            /* C */ { 25, 0, 0, 0},
            /* D */ {  0, 0, 0, 0}};
        static int scholarship_pct(int marks, int income)
        {
            char g = grade_table(marks);
            int gi = grade_index(g);
            return gi < 0 ? 0 : scholarship[gi][income_band(income)];
        }

        int main(void)
        {
            int tests[] = {100, 90, 89, 80, 79, 70, 69, 60, 59, 50, 49, 0, -1, 101};
            int nt = sizeof tests / sizeof tests[0];
            printf("%-6s %-7s %-7s %-7s %-9s\n", "marks", "ladder", "switch", "table", "no-break");
            for (int i = 0; i < nt; i++) {
                int m = tests[i];
                printf("%-6d %-7c %-7c %-7c %-9c\n", m, grade_ladder(m), grade_switch(m), grade_table(m), grade_switch_nobreak(m)
            }

            int mism = 0, mism_nb = 0;
            cmp_ladder = 0;
            for (int m = -5; m <= 105; m++) {
                char a = grade_ladder(m), b = grade_switch(m), c = grade_table(m);
                if (a != b || a != c) mism++;
                if (a != grade_switch_nobreak(m)) mism_nb++;
            }
            printf("\nExhaustive -5..105 (111 values): ladder/switch/table mismatches = %d; no-break mismatches = %d\n", mism, mi

            unsigned long total = 0, n = 0;
            for (int m = 0; m <= 100; m++) { cmp_ladder = 0; grade_ladder(m); total += cmp_ladder; n++; }
            printf("Ladder comparisons for marks 0..100: total=%lu average=%.2f per grade (max 6)\n", total, (double)total / n);
            printf("Switch: 1 range check + 1 jump-table index; table: 1 range check + 1 index (O(1))\n");

            printf("\nScholarship rule (marks, income) -> %%\n");
            int sm[] = {95, 85, 75, 65, 55, 45}; int si[] = {200000, 400000, 600000, 900000};
            printf("marks \\ income  200000 400000 600000 900000\n");
            for (int i = 0; i < 6; i++) {
                printf("%-6d (%c)       ", sm[i], grade_table(sm[i]));
                for (int j = 0; j < 4; j++) printf("%-7d", scholarship_pct(sm[i], si[j]));
                printf("\n");
            }
            return 0;
        }
```

Q4/q4_complete.c

```
        #include <stdio.h>
        /* Grade rules: 90-100 S, 80-89 A, 70-79 B, 60-69 C, 50-59 D, below 50 F, invalid '?' */
        char grade(int marks)
        {
            if (marks < 0 || marks > 100) return '?';
            /* TODO 1: switch on marks / 10, combining cases where possible */
```

```
            switch (marks / 10) {
            case 10:
            case 9:  return 'S';
            case 8:  return 'A';
            case 7:  return 'B';
            case 6:  return 'C';
            case 5:  return 'D';
            default: return 'F';   /* 0..49 -> marks/10 in 0..4 */
            }
        }

        int main(void)
        {
            int marks[] = {100, 95, 83, 50, 49, -5};
            for (int i = 0; i < 6; i++) printf("%d:%c ", marks[i], grade(marks[i]));
            printf("\n");
            return 0;
        }
```

Output: `Q4/output_compare.txt`

```
marks  ladder  switch  table   no-break
100    S       S       S       S
90     S       S       S       S
89     A       A       A       B
80     A       A       A       B
79     B       B       B       B
70     B       B       B       B
69     C       C       C       C
60     C       C       C       C
59     D       D       D       D
50     D       D       D       D
49     F       F       F       F
0      F       F       F       F
-1     ?       ?       ?       ?
101    ?       ?       ?       ?

Exhaustive -5..105 (111 values): ladder/switch/table mismatches = 0; no-break mismatches = 10
Ladder comparisons for marks 0..100: total=502 average=4.97 per grade (max 6)
Switch: 1 range check + 1 jump-table index; table: 1 range check + 1 index (O(1))

Scholarship rule (marks, income) -> %
marks \ income  200000 400000 600000 900000
95     (S)       100    75     50     25
85     (A)       75     50     25     0
75     (B)       50     25     0      0
65     (C)       25     0      0      0
55     (D)       0      0      0      0
45     (F)       0      0      0      0
```

Output: `Q4/output_complete.txt`

```
100:S 95:S 83:A 50:D 49:F -5:?
```

Q5

`Q5/q5_complete.c`

```
        #include <stdio.h>
        unsigned long checks = 0; /* number of divisions performed by the last call */

        /* Returns 1 if n is prime, otherwise 0. */
        int is_prime(unsigned long n)
        {
            checks = 0;
            if (n < 2) return 0;
            /* TODO 1: try i = 2,3,4... while i*i <= n, count each division, return 0 on divisor */
            for (unsigned long i = 2; i * i <= n; i++) {
                checks++;
                if (n % i == 0) return 0;
            }
            return 1;
        }

        int main(void)
        {
            unsigned long tickets[] = {97, 100, 7919, 1000003};
            for (int k = 0; k < 4; k++) {
                int p = is_prime(tickets[k]);
                printf("%8lu %-9s divisions = %lu\n", tickets[k],
                       p ? "prime" : "not prime", checks);
            }
```

```
    return 0;
}
```

Q5/q5_versions.c

```
/* Q5: all loop versions, division counts, boundary cases, overflow, sieve */
#include <stdio.h>
#include <stdint.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>

unsigned long checks = 0;

static int prime_full_for(unsigned long n)          /* divisors 2 .. n-1 */
{
    checks = 0; if (n < 2) return 0;
    for (unsigned long i = 2; i < n; i++) { checks++; if (n % i == 0) return 0; }
    return 1;
}
static int prime_sqrt_for(unsigned long n)
{
    checks = 0; if (n < 2) return 0;
    for (unsigned long i = 2; i * i <= n; i++) { checks++; if (n % i == 0) return 0; }
    return 1;
}
static int prime_sqrt_while(unsigned long n)
{
    checks = 0; if (n < 2) return 0;
    unsigned long i = 2;
    while (i * i <= n) { checks++; if (n % i == 0) return 0; i++; }
    return 1;
}
/* Correct do-while: the guard for n < 4 must come BEFORE the loop */
static int prime_sqrt_dowhile(unsigned long n)
{
    checks = 0; if (n < 2) return 0;
    if (n < 4) return 1;                /* 2 and 3: loop body would test 2 % 2 */
    unsigned long i = 2;
    do { checks++; if (n % i == 0) return 0; i++; } while (i * i <= n);
    return 1;
}
/* Naive do-while without the guard (shows the bug) */
static int prime_dowhile_naive(unsigned long n)
{
    checks = 0; if (n < 2) return 0;
    unsigned long i = 2;
    do { checks++; if (n % i == 0) return 0; i++; } while (i * i <= n);
    return 1;
}

/* Overflow demo with 32-bit arithmetic */
static int prime_u32_unsafe(uint32_t n, unsigned long limit, int *hit_limit)
{
    unsigned long it = 0; *hit_limit = 0;
    for (uint32_t i = 2; i * i <= n; i++) { if (++it > limit) { *hit_limit = 1; return -1; } if (n % i == 0) return 0; }
    return 1;
}
static int prime_u32_safe(uint32_t n)
{
    for (uint32_t i = 2; i <= n / i; i++) if (n % i == 0) return 0;   /* no multiplication */
    return 1;
}

int main(void)
{
    unsigned long t[] = {2, 3, 4, 97, 100, 7919, 1000003, 999983, 1000001};
    int nt = sizeof t / sizeof t[0];
    printf("%-9s %-9s | %10s %10s %10s %10s\n", "n", "result", "i<n (for)", "sqrt for", "sqrt while", "sqrt do-wh");
    for (int k = 0; k < nt; k++) {
        unsigned long n = t[k];
        int r1 = prime_full_for(n);  unsigned long c1 = checks;
        int r2 = prime_sqrt_for(n);  unsigned long c2 = checks;
        int r3 = prime_sqrt_while(n);unsigned long c3 = checks;
        int r4 = prime_sqrt_dowhile(n); unsigned long c4 = checks;
        printf("%-9lu %-9s | %10lu %10lu %10lu %10lu %s\n", n, r2 ? "prime" : "not prime", c1, c2, c3, c4,
               (r1 == r2 && r2 == r3 && r3 == r4) ? "" : "<-- MISMATCH");
    }

    printf("\ndo-while without guard: n=2 -> %s, n=3 -> %s (wrong: 2%%2==0 on first pass)\n",
           prime_dowhile_naive(2) ? "prime" : "NOT prime", prime_dowhile_naive(3) ? "prime" : "NOT prime");
    printf("do-while with guard   : n=2 -> %s, n=3 -> %s\n",
           prime_sqrt_dowhile(2) ? "prime" : "NOT prime", prime_sqrt_dowhile(3) ? "prime" : "NOT prime");

    int hit;
```

```
            uint32_t big = 4294967291u; /* largest prime below 2^32 */
            int r = prime_u32_unsafe(big, 100000UL, &hit);
            printf("\nOverflow: uint32 n=%u, loop 'i*i <= n' : result=%d, stopped by safety limit=%s (i*i wraps to 0 at i=65536)\
                   big, r, hit ? "YES" : "no");
            printf("          safe condition 'i <= n/i' : %s\n", prime_u32_safe(big) ? "prime (correct)" : "not prime");

            /* Sieve comparison on N = 1,000,003 */
            enum { N = 1000003 };
            unsigned char *flag = malloc(N + 1);
            memset(flag, 1, N + 1); flag[0] = flag[1] = 0;
            unsigned long marks = 0;
            clock_t c0 = clock();
            for (unsigned long i = 2; i * i <= N; i++)
                if (flag[i]) for (unsigned long j = i * i; j <= N; j += i) { flag[j] = 0; marks++; }
            clock_t c1 = clock();
            unsigned long np = 0; for (int i = 0; i <= N; i++) np += flag[i];
            printf("\nSieve up to %d: %lu marking operations, %lu primes, memory %d bytes, time %.2f ms\n",
                   N, marks, np, N + 1, 1000.0 * (c1 - c0) / CLOCKS_PER_SEC);
            printf("Sieve lookups: 97->%d, 100->%d, 7919->%d, 1000003->%d (0 divisions each)\n", flag[97], flag[100], flag[7919],
flag[1000003]);

            unsigned long total = 0;
            for (unsigned long n = 2; n <= N; n++) { prime_sqrt_for(n); total += checks; }
            printf("Trial division of ALL n in 2..%d: total divisions = %lu\n", N, total);
            free(flag);
            return 0;
        }
```

Output: `Q5/output_complete.txt`

```
      97 prime     divisions = 8
     100 not prime divisions = 1
    7919 prime     divisions = 87
 1000003 prime     divisions = 999
```

Output: `Q5/output_versions.txt`

```
n         result    |  i<n (for)   sqrt for sqrt while sqrt do-wh
2         prime     |          0          0          0          0
3         prime     |          1          0          0          0
4         not prime |          1          1          1          1
97        prime     |         95          8          8          8
100       not prime |          1          1          1          1
7919      prime     |       7917         87         87         87
1000003   prime     |    1000001        999        999        999
999983    prime     |     999981        998        998        998
1000001   not prime |        100        100        100        100

do-while without guard: n=2 -> NOT prime, n=3 -> prime (wrong: 2%2==0 on first pass)
do-while with guard   : n=2 -> prime, n=3 -> prime

Overflow: uint32 n=4294967291, loop 'i*i <= n' : result=-1, stopped by safety limit=YES (i*i wraps to 0 at i=65536)
          safe condition 'i <= n/i' : prime (correct)

Sieve up to 1000003: 2122051 marking operations, 78499 primes, memory 1000004 bytes, time 4.86 ms
Sieve lookups: 97->1, 100->0, 7919->1, 1000003->1 (0 divisions each)
Trial division of ALL n in 2..1000003: total divisions = 67741504
```

Q6

`Q6/q6_loan.c`

```
        /* Q6 - Self-help-group micro-loan schedule, termination, growth, closed-form */
        #include <stdio.h>
        #include <math.h>

        #define MAX_MONTHS 360
        #define EPS 0.005   /* half a paisa */

        /* Returns months taken, or -1 if loan never clears. Prints schedule when verbose. */
        static int schedule(double principal, double rate, double pay, int verbose, double *total_interest)
        {
            double balance = principal, interest_sum = 0.0;
            int month = 0;
            *total_interest = 0.0;

            if (pay <= principal * rate) {      /* repayment cannot even cover first month's interest */
                if (verbose) printf("  REJECTED: repayment %.2f <= first-month interest %.2f; balance would never fall.\n", pay,
principal * rate);
                return -1;
            }
            if (verbose) printf("  %-5s %-10s %-10s %-10s %-10s\n", "Month", "Opening", "Interest", "Payment", "Closing");
```

```c
            while (balance > EPS) {             /* while: number of months unknown in advance */
                if (month >= MAX_MONTHS) {      /* safety limit */
                    if (verbose) printf("  STOPPED: reached %d months with balance %.2f still outstanding.\n", MAX_MONTHS, balanc
                    *total_interest = interest_sum;
                    return -1;
                }
                double opening = balance;
                double interest = balance * rate;
                balance += interest;
                double payment = (balance < pay) ? balance : pay;   /* last month pays only what remains */
                balance -= payment;
                interest_sum += interest;
                month++;
                if (verbose) printf("  %-5d %-10.2f %-10.2f %-10.2f %-10.2f\n", month, opening, interest, payment, balance < EPS
balance);
            }
            *total_interest = interest_sum;
            return month;
        }

        int main(void)
        {
            double ti;
            printf("Loan Rs 10000 at 2%% per month, repayment Rs 2000\n");
            int m = schedule(10000.0, 0.02, 2000.0, 1, &ti);
            printf("  Months = %d, total interest = %.4f, total paid = %.4f\n", m, ti, 10000.0 + ti);

            /* Closed form */
            double r = 0.02, P = 10000.0, E = 2000.0;
            double nf = -log(1.0 - r * P / E) / log(1.0 + r);
            printf("  Closed form n = %.4f -> ceil = %d months\n", nf, (int)ceil(nf));

            /* Why balance != 0 is dangerous */
            printf("\nDanger of 'balance != 0' with floating point (no last-month adjustment, limit 400):\n");
            double b = 10000.0; int k = 0;
            while (b != 0 && k < 400) { b = b * 1.02 - 2000.0; k++; if (b < 0) break; }
            printf("  After %d months balance = %.6f (never exactly 0; went negative = loan 'overpaid')\n", k, b);
            b = 0.1; k = 0; while (b != 1.0 && k < 20) { b += 0.1; k++; }
            printf("  Adding 0.1 repeatedly: reached 1.0 exactly? %s (after %d steps b=%.17g)\n", b == 1.0 ? "yes" : "NO", k, b);

            /* Growth investigation */
            printf("\nGrowth investigation (P=10000, r=2%%, monthly interest = 200):\n");
            printf("  %-10s %-8s %-12s %-12s %-10s\n", "repay", "months", "closed-form", "interest", "status");
            double pays[] = {5000, 3000, 2000, 1000, 500, 300, 250, 220, 210, 205, 201, 200.01, 200, 150};
            for (int i = 0; i < (int)(sizeof pays / sizeof pays[0]); i++) {
                int mm = schedule(10000.0, 0.02, pays[i], 0, &ti);
                double cf = (pays[i] > 200.0) ? -log(1.0 - 0.02 * 10000.0 / pays[i]) / log(1.02) : -1;
                if (mm > 0) printf("  %-10.2f %-8d %-12.2f %-12.2f ok\n", pays[i], mm, cf, ti);
                else if (pays[i] <= 200.0) printf("  %-10.2f %-8s %-12s %-12s rejected (repay <= interest)\n", pays[i], "-", "-",
                else printf("  %-10.2f %-8s %-12.2f %-12.2f hit %d-month limit\n", pays[i], ">360", cf, ti, MAX_MONTHS);
            }

            /* Deployment decision: limit message */
            printf("\nSafety limit demo (repay Rs 200.01):\n");
            schedule(10000.0, 0.02, 200.01, 0, &ti);
            m = schedule(10000.0, 0.02, 200.01, 0, &ti);
            if (m < 0) printf("  Message: loan not repayable within %d months at this instalment; increase repayment.\n", MAX_MON
            printf("Rejected case (repay Rs 150):\n");
            schedule(10000.0, 0.02, 150.0, 1, &ti);
            return 0;
        }
```

Output: `Q6/output.txt`

```
Loan Rs 10000 at 2% per month, repayment Rs 2000
  Month Opening    Interest   Payment    Closing
  1     10000.00   200.00     2000.00    8200.00
  2     8200.00    164.00     2000.00    6364.00
  3     6364.00    127.28     2000.00    4491.28
  4     4491.28    89.83      2000.00    2581.11
  5     2581.11    51.62      2000.00    632.73
  6     632.73     12.65      645.38     0.00
  Months = 6, total interest = 645.3823, total paid = 10645.3823
  Closed form n = 5.3205 -> ceil = 6 months

Danger of 'balance != 0' with floating point (no last-month adjustment, limit 400):
  After 6 months balance = -1354.617734 (never exactly 0; went negative = loan 'overpaid')
  Adding 0.1 repeatedly: reached 1.0 exactly? NO (after 20 steps b=2.1000000000000005)

Growth investigation (P=10000, r=2%, monthly interest = 200):
  repay      months   closed-form  interest     status
  5000.00    3        2.06         310.08       ok
  3000.00    4        3.48         459.50       ok
  2000.00    6        5.32         645.38       ok
  1000.00    12       11.27        1270.33      ok
```

```
  500.00     26       25.80        2898.73      ok
  300.00     56       55.48        6644.17      ok
  250.00     82       81.27        10318.98     ok
  220.00     122      121.09       16639.92     ok
  210.00     154      153.74       22286.50     ok
  205.00     188      187.53       28444.00     ok
  201.00     268      267.81       43829.75     ok
  200.01     >360     500.11       71380.32     hit 360-month limit
  200.00     -        -            -            rejected (repay <= interest)
  150.00     -        -            -            rejected (repay <= interest)

Safety limit demo (repay Rs 200.01):
  Message: loan not repayable within 360 months at this instalment; increase repayment.
Rejected case (repay Rs 150):
  REJECTED: repayment 150.00 <= first-month interest 200.00; balance would never fall.
```

Q7

Q7/q7_redesign.c

```c
/* Q7 - Day-4 Design Challenge: redesign of Q5 prime checking.
   Original  : trial division by every i with i*i <= n         (O(sqrt n) per query)
   Redesign 1: skip multiples of 2 and 3 (6k +/- 1 wheel)      (about sqrt(n)/3 per query, no extra memory)
   Redesign 2: sieve once, answer each query by lookup         (O(N log log N) once, O(1) per query, O(N) memory) */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>

unsigned long checks = 0;

static int prime_orig(unsigned long n)
{
    checks = 0; if (n < 2) return 0;
    for (unsigned long i = 2; i * i <= n; i++) { checks++; if (n % i == 0) return 0; }
    return 1;
}

static int prime_wheel(unsigned long n)
{
    checks = 0;
    if (n < 2) return 0;
    if (n < 4) return 1;                        /* 2, 3 */
    checks++; if (n % 2 == 0) return 0;
    checks++; if (n % 3 == 0) return 0;
    for (unsigned long i = 5; i * i <= n; i += 6) {   /* i = 6k-1, i+2 = 6k+1 */
        checks++; if (n % i == 0) return 0;
        checks++; if (n % (i + 2) == 0) return 0;
    }
    return 1;
}

static unsigned char *flag; static unsigned long sieve_ops = 0;
static void build_sieve(unsigned long N)
{
    flag = malloc(N + 1); memset(flag, 1, N + 1); flag[0] = flag[1] = 0; sieve_ops = 0;
    for (unsigned long i = 2; i * i <= N; i++)
        if (flag[i]) for (unsigned long j = i * i; j <= N; j += i) { flag[j] = 0; sieve_ops++; }
}

int main(void)
{
    unsigned long tickets[] = {97, 100, 7919, 1000003, 999983, 1000001};
    int nt = sizeof tickets / sizeof tickets[0];

    printf("A. Same inputs, divisions per query\n");
    printf("%-9s %-9s %10s %10s %8s %10s\n", "n", "result", "original", "6k+-1", "saved", "sieve");
    for (int k = 0; k < nt; k++) {
        unsigned long n = tickets[k];
        int a = prime_orig(n); unsigned long ca = checks;
        int b = prime_wheel(n); unsigned long cb = checks;
        printf("%-9lu %-9s %10lu %10lu %7.1f%% %10d%s\n", n, a ? "prime" : "not prime", ca, cb,
               ca ? 100.0 * (double)(ca - cb) / ca : 0.0, 0, a == b ? "" : "  MISMATCH");
    }

    const unsigned long N = 1000003;
    clock_t s0 = clock(); build_sieve(N); clock_t s1 = clock();
    printf("\nB. Batch workload: verify EVERY ticket 1..%lu\n", N);
    unsigned long tot_o = 0, tot_w = 0, bad = 0, nprimes = 0;
    clock_t t0 = clock();
    for (unsigned long n = 1; n <= N; n++) { prime_orig(n); tot_o += checks; }
    clock_t t1 = clock();
    for (unsigned long n = 1; n <= N; n++) { int w = prime_wheel(n); tot_w += checks; if (w != flag[n]) bad++; nprimes +=
    clock_t t2 = clock();
```

```
            unsigned long hits = 0;
            clock_t t3 = clock();
            for (unsigned long n = 1; n <= N; n++) hits += flag[n];
            clock_t t4 = clock();
            printf("  %-22s divisions=%-10lu time=%7.1f ms  memory=0 bytes\n", "original (sqrt)", tot_o, 1000.0 * (t1 - t0) /
CLOCKS_PER_SEC);
            printf("  %-22s divisions=%-10lu time=%7.1f ms  memory=0 bytes\n", "6k+-1 wheel", tot_w, 1000.0 * (t2 - t1) /
CLOCKS_PER_SEC);
            printf("  %-22s divisions=0          marks=%lu build=%.1f ms + lookups %.1f ms memory=%lu bytes\n", "sieve", sieve_op
                  1000.0 * (s1 - s0) / CLOCKS_PER_SEC, 1000.0 * (t4 - t3) / CLOCKS_PER_SEC, N + 1);
            printf("  Primes found: wheel=%lu sieve=%lu, disagreements between wheel and sieve = %lu\n", nprimes, hits, bad);
            printf("  Division reduction wheel vs original: %.1f%%\n", 100.0 * (double)(tot_o - tot_w) / tot_o);

            printf("\nC. Few queries (break-even): sieve cost %lu marks vs trial division per query\n", sieve_ops);
            unsigned long qs[] = {1, 10, 100, 1000, 10000};
            for (int i = 0; i < 5; i++) {
                unsigned long q = qs[i], tw = 0, to = 0;
                for (unsigned long j = 0; j < q; j++) { unsigned long n = N - j * 2; prime_orig(n); to += checks; prime_wheel(n);
checks; }
                printf("  %-6lu queries (large n near %lu): original=%-9lu wheel=%-9lu sieve=%lu marks\n", q, N, to, tw, sieve_op
            }
            free(flag);
            return 0;
        }
```

Output: `Q7/output.txt`

```
A. Same inputs, divisions per query
n         result      original      6k+-1    saved      sieve
97        prime              8          4    50.0%          0
100       not prime          1          1     0.0%          0
7919      prime             87         30    65.5%          0
1000003   prime            999        334    66.6%          0
999983    prime            998        334    66.5%          0
1000001   not prime        100         35    65.0%          0

B. Batch workload: verify EVERY ticket 1..1000003
  original (sqrt)        divisions=67741504   time=  180.1 ms  memory=0 bytes
  6k+-1 wheel            divisions=23739568   time=   58.6 ms  memory=0 bytes
  sieve                  divisions=0          marks=2122051 build=5.0 ms + lookups 1.7 ms memory=1000004 bytes
  Primes found: wheel=78499 sieve=78499, disagreements between wheel and sieve = 0
  Division reduction wheel vs original: 65.0%

C. Few queries (break-even): sieve cost 2122051 marks vs trial division per query
  1      queries (large n near 1000003): original=999       wheel=334       sieve=2122051 marks
  10     queries (large n near 1000003): original=1903      wheel=650       sieve=2122051 marks
  100    queries (large n near 1000003): original=18925     wheel=6468      sieve=2122051 marks
  1000   queries (large n near 1000003): original=186501    wheel=63753     sieve=2122051 marks
  10000  queries (large n near 1000003): original=1872978   wheel=640850    sieve=2122051 marks
```